

Reviewer Pack — Roe-trace pairing for Inner-Product realizes coarse depth κ ...

Entry 738bb754804a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 12:41:08 UTC

Roe-trace pairing for Inner-Product realizes coarse depth $\kappa \geq \log(R)$ and certifies a quadratic-log gap against random f

Entry ID: 738bb754804a **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 12:41:08 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Karchmer-Wigderson (CG-KW) — coarse geometry / Roe-algebraic K-theory applied to circuit complexity (mathematical branch: noncommutative geometry & geometric group theory) **Field B** (complexity object): Boolean formula depth (NC^1 vs. P): explicit super-logarithmic depth lower bounds via index pairings on the KW-metric space

Statement:

Let $f = IP_n : \{0,1\}^{2n} \rightarrow \{0,1\}$ be the inner-product-mod-2 function and let (X_f, d_f) be its KW-metric space with $d_f(x,y) := \log_2(\text{min number of coordinates needed to distinguish } x,y)$. Let $c_{IP} \in HX^1(X_f)$ be the coarse cocycle obtained from the canonical ‘parity-of-disagreement’ 1-cochain $\bar{c}_{IP}(x,y) = \langle x \oplus y, x' \oplus y' \rangle \bmod 2$ (lifted to $\mathbb{Z}$ via signed indicator), and for each propagation parameter $R \geq 1$ let $T_R \in C_u^*[X_f]$ be the banded Roe operator $T_R(x,y) := 1[d_f(x,y) \leq R]/(1+|B_R(x)|)$. Then (i) (axiom A5 instance) the trace pairing satisfies $\log| \langle c_{IP}, T_R \rangle | / \log R \rightarrow \alpha \geq 1$ as $n \rightarrow \infty$ and $R = o(n)$, so $\kappa(IP_n) \geq 1$ with this normalization; (ii) (axiom A3 instance) for a uniformly random f' on the same $X_{\{f'\}}$ the same construction yields $\log| \langle c_{IP}, T_R \rangle | / \log R \rightarrow 0$ in probability, separating structured and random f .

Rationale (proposer’s reasoning):

This is a direct empirical probe of axioms A5 (Roe-index rigidity: explicit candidates realize a nontrivial pairing whose growth rate is the coarse depth κ) and A3 (anti-natural-proofs: random instances yield trivial pairings). IP_n is the smallest ‘KRW-friendly’ explicit function whose KW protocol has a clean parity structure that lifts to a coarse cocycle, so we expect the pairing’s logarithmic-growth exponent to be a faithful proxy for κ . Sub-additivity (A2) and Coarse-KW link (A1) both predict $\kappa(IP_n) \geq 1$ in this normalization (a sub-claim of the conjectured $\Omega(\log^2 n)$ for Andreev), making a positive numerical exponent a falsifiable prediction. A null result would either refute A5 for IP or indicate the normalization choice for d_f is too coarse — both informative.

Taxonomy category: KARCHMER_WIGDERSON (status at proposal time:)

Framework membership: framework fw_85a254b4a0, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7fe056329997e21a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For $n \in \{3, 4, 5, 6\}$, fit slope $\alpha(n)$ of $\log|p(R)|$ vs $\log R$ for IP_n , and $\alpha'(n)$ as mean over ≥ 20 random- f' seeds. SUPPORT requires $\alpha(n) \geq 0.7$ for $n \in \{5, 6\}$, $\alpha(n)$ non-decreasing in n , and $\alpha'(n) \leq 0.2$ with 95% CI upper bound < 0.4 for all n .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Roe algebra coarse cohomology Karchmer-Wigderson circuit depth lower bound - inner product mod 2 formula depth coarse geometry index pairing - uniform Roe C^* -algebra trace cocycle Boolean function complexity

Top relevant hits considered: - [<http://arxiv.org/abs/2002.07444v1>] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [<http://arxiv.org/abs/2107.05128v2>] Karchmer-Wigderson Games for Hazard-free Computation - [<http://arxiv.org/abs/1404.7443v2>] Depth Lower Bounds against Circuits with Sparse Orientation - [<http://arxiv.org/abs/1011.5800v2>] Operator-valued pseudo-differential operators and the twisted index pairing - [<http://arxiv.org/abs/0710.5926v2>] Mod 2 cohomology of 2-local finite groups of low rank - [<http://arxiv.org/abs/1803.07099v3>] Varieties of coarse spaces - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/1407.6169v3>] Multiplicative Complexity of Vector Valued Boolean Functions

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import math
import random

def hamming_weight(x):
    return bin(x).count('1')

def xor(x, y):
    return x ^ y
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 8, 11, 14]
    results = []

    for n in n_values:
        X_f = [(i << n) | j for i in range(1 << n) for j in range(1 << n)]
        f = lambda x: xor(x >> n, x & ((1 << n) - 1))

        d_f = [[math.log2(hamming_weight(xor(x, y))) for y in X_f] for x in X_f]
        c_IP = [[(-1) ** f(xor(x, y)) if d_f[x][y] <= math.log2(2 * n) else 0 for y in range(len(X_f))] for x in range(len(X_f))]

        p_R_values = []
        for R in range(1, int(math.log2(2 * n)) + 1):
            T_R = [[1 if d_f[x][y] <= R else 0 for y in range(len(X_f))] for x in range(len(X_f))]
            ball_volume = sum(sum(row) for row in T_R)
            T_R = [[t / ball_volume for t in row] for row in T_R]
            p_R = sum(c_IP[x][y] * T_R[x][y] for x in range(len(X_f)) for y in range(len(X_f)))
            p_R_values.append(math.log(abs(p_R)))

        alpha_n = (p_R_values[-1] - p_R_values[0]) / math.log2(2 * n)
        results.append({"n": n, "alpha_n": alpha_n})

    return {
        "metric_name": "alpha_n",
        "metric_value": sum(result["alpha_n"] for result in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": all(result["alpha_n"] >= 0.7 for result in results),
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d19f60d5.py", line 60, in <module>
    result = run_trial(seed)
              ~~~~~
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_d19f60d5.py", line 34, in run_trial
    d_f = [[math.log2(hamming_weight(xor(x, y))) for y in X_f] for x in X_f]
           ~~~~~
ValueError: math domain error

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a math domain error (log2 of zero when $x == y$) before producing any $\alpha(n)$ or $\alpha'(n)$ data, so neither the support nor falsification conditions can be evaluated.
| next: Guard the d_f computation against the diagonal (e.g., set $d_f(x,x) := 0$ or skip $x=y$ pairs) and re-run for $n \in \{3,4,5,6\}$ with ≥ 20 random-f' seeds.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	8164
2	novelty	claude_max	opus	0	0	4613
3	novelty	claude_max	opus	0	0	10401
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16024
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6382
6	judge	claude_max	opus	0	0	5447

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 51033 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/738bb754804a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/738bb754804a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/738bb754804a.tar.gz` (if generated)